

Proyecto Algoritmo de optimización

Nombre : Edwin Bolivar

URL: <https://github.com/ebogo2025/AlgoritmosOptimizacion.git>

Problema: Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible.

```
import pulp
!pip install pulp

#
=====
# PASO 1: INGRESAR LA MATRIZ DE DATOS (30 Tomas x 10 Actores)
#
=====
# Cada sub-lista [ ] representa una TOMA (del 1 al 30).
# Dentro de cada sub-lista hay 10 posiciones (una por cada ACTOR del 1 al 10).
matriz_tomas_actores = [
    [1, 1, 1, 1, 1, 0, 0, 0, 0, 0], # Toma 1 (Actúan: Actores 1, 2, 3, 4, 5)
    [0, 0, 1, 1, 1, 0, 0, 0, 0, 0], # Toma 2 (Actúan: Actores 3, 4, 5)
    [0, 1, 0, 0, 1, 0, 1, 0, 0, 0], # Toma 3 (Actúan: Actores 2, 5, 7)
    [1, 1, 0, 0, 0, 0, 1, 1, 0, 0], # Toma 4 (Actúan: Actores 1, 2, 7, 8)
    [0, 1, 0, 1, 0, 0, 0, 1, 0, 0], # Toma 5 (Actúan: Actores 2, 4, 8)
    [1, 1, 0, 1, 1, 0, 0, 0, 0, 0], # Toma 6 (Actúan: Actores 1, 2, 4, 5)
    [1, 1, 0, 1, 1, 0, 0, 0, 0, 0], # Toma 7 (Actúan: Actores 1, 2, 4, 5)
    [1, 1, 0, 0, 0, 1, 0, 0, 0, 0], # Toma 8 (Actúan: Actores 1, 2, 6)
    [1, 1, 0, 1, 0, 0, 0, 0, 0, 0], # Toma 9 (Actúan: Actores 1, 2, 4)
    [1, 1, 0, 0, 0, 1, 0, 0, 1, 0], # Toma 10 (Actúan: Actores 1, 2, 6, 9)
    [1, 1, 1, 0, 1, 0, 0, 1, 0, 0], # Toma 11 (Actúan: Actores 1, 2, 3, 5, 8)
    [1, 1, 1, 1, 0, 1, 0, 0, 0, 0], # Toma 12 (Actúan: Actores 1, 2,
```

```

3, 4, 6)
    [1, 0, 0, 1, 1, 0, 0, 0, 0, 0], # Toma 13 (Actúan: Actores 1, 4,
5)
    [1, 0, 1, 0, 0, 1, 0, 0, 0, 0], # Toma 14 (Actúan: Actores 1, 3,
6)
    [1, 1, 0, 0, 0, 0, 1, 0, 0, 0], # Toma 15 (Actúan: Actores 1, 2,
7)
    [0, 0, 0, 1, 0, 0, 0, 0, 0, 1], # Toma 16 (Actúan: Actores 4, 10)
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # Toma 17 (Actúan: Actores 1, 3)
    [0, 0, 1, 0, 0, 1, 0, 0, 0, 0], # Toma 18 (Actúan: Actores 3, 6)
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # Toma 19 (Actúan: Actores 1, 3)
    [1, 0, 1, 1, 1, 0, 0, 0, 0, 0], # Toma 20 (Actúan: Actores 1, 3,
4, 5)
    [0, 0, 0, 0, 0, 1, 0, 1, 0, 0], # Toma 21 (Actúan: Actores 6, 8)
    [1, 1, 1, 1, 0, 0, 0, 0, 0, 0], # Toma 22 (Actúan: Actores 1, 2,
3, 4)
    [1, 0, 1, 0, 0, 0, 0, 0, 0, 0], # Toma 23 (Actúan: Actores 1, 3)
    [0, 0, 1, 0, 0, 1, 0, 0, 0, 0], # Toma 24 (Actúan: Actores 3, 6)
    [1, 1, 0, 1, 0, 0, 0, 0, 0, 1], # Toma 25 (Actúan: Actores 1, 2,
4, 10)
    [1, 0, 1, 0, 1, 0, 0, 0, 1, 0], # Toma 26 (Actúan: Actores 1, 3,
5, 9)
    [0, 0, 0, 1, 1, 0, 0, 0, 0, 0], # Toma 27 (Actúan: Actores 4, 5)
    [1, 0, 0, 1, 0, 0, 0, 0, 0, 0], # Toma 28 (Actúan: Actores 1, 4)
    [1, 0, 0, 0, 1, 1, 0, 0, 0, 0], # Toma 29 (Actúan: Actores 1, 5,
6)
    [1, 0, 0, 1, 0, 0, 0, 0, 0, 0], # Toma 30 (Actúan: Actores 1, 4)
]

```

Definimos los totales fijos del problema

NUM_ACTORES = 10

NUM_TOMAS = 30

MAX_TOMAS_POR_DIA = 6

MAX_DIAS = 5 *# 30 tomas divididas entre 6 por día = 5 días exactos*

Requirement already satisfied: pulp in c:\users\edwin\anaconda3\lib\site-packages (3.3.2)

#

=====

=====

PASO 2: CREAR EL CONTENEDOR DEL PROBLEMA MATEMÁTICO

#

=====

=====

Le llamamos "prob" y le indicamos "pulp.LpMinimize" porque queremos PAGAR MENOS.

prob = pulp.LpProblem("Planificacion_Doblaje", pulp.LpMinimize)

#

=====

```

=====
# PASO 3: CREAR LAS VARIABLES DE DECISIÓN (SI/NO)
#
=====
=====

# Variable x: Indica si la TOMA 'i' se graba el DÍA 'd' (0 = No, 1 = Sí)
x = pulp.LpVariable.dicts(
    "Grabar_Toma",
    ((i, d) for i in range(NUM_TOMAS) for d in range(MAX_DIAS)),
    cat=pulp.LpBinary
)

# Variable y: Indica si el ACTOR 'a' asiste a trabajar el DÍA 'd' (0 = No, 1 = Sí)
y = pulp.LpVariable.dicts(
    "Asistir_Actor",
    ((a, d) for a in range(NUM_ACTORES) for d in range(MAX_DIAS)),
    cat=pulp.LpBinary
)

#
=====
=====
# PASO 4: DEFINIR LO QUE QUEREMOS MINIMIZAR (FUNCIÓN OBJETIVO)
#
=====
=====
# Queremos que la suma total de asistencias de todos los actores en los 5 días sea la mínima posible.
prob += pulp.lpSum(y[a, d] for a in range(NUM_ACTORES) for d in range(MAX_DIAS))

#
=====
=====
# PASO 5: AÑADIR LAS REGLAS (RESTRICCIONES)
#
=====
=====

# REGLA 1: Cada toma debe grabarse en exactamente UN día.
for i in range(NUM_TOMAS):
    prob += pulp.lpSum(x[i, d] for d in range(MAX_DIAS)) == 1

# REGLA 2: No se pueden grabar más de 6 tomas por día.
for d in range(MAX_DIAS):

```

```

    prob += pulp.lpSum(x[i, d] for i in range(NUM_TOMAS)) <=
MAX_TOMAS_POR_DIA

# REGLA 3: Si una toma se graba el día 'd', sus actores TIENEN que ir
a trabajar ese día.
for i in range(NUM_TOMAS):
    for a in range(NUM_ACTORES):
        # Si la matriz dice que el actor 'a' actúa en la toma 'i' (es
decir, vale 1)
        if matriz_tomas_actores[i][a] == 1:
            for d in range(MAX_DIAS):
                # La asistencia del actor (y) debe ser mayor o igual
que la grabación de la toma (x)
                prob += y[a, d] >= x[i, d]

#
=====
=====
# PASO 6: RESOLVER Y MOSTRAR LA SOLUCIÓN
#
=====
=====
# Le pedimos al programa que busque la combinación óptima
prob.solve(pulp.PULP_CBC_CMD(msg=False))

print(f"Estado de la solución: {pulp.LpStatus[prob.status]}")
print(f"Costo Mínimo Total (Total de jornales/días pagados):
{int(pulp.value(prob.objective))}\n")

# Mostramos el calendario final día por día
for d in range(MAX_DIAS):
    # Buscamos qué tomas tienen un valor de 1 en el Día 'd'
    tomas_del_dia = [i + 1 for i in range(NUM_TOMAS) if
pulp.value(x[i, d]) == 1]

    # Buscamos qué actores tienen un valor de 1 en el Día 'd'
    actores_del_dia = [a + 1 for a in range(NUM_ACTORES) if
pulp.value(y[a, d]) == 1]

    print(f"=== DÍA {d + 1} ===")
    print(f"  Tomas a filmar ({len(tomas_del_dia)}/6):
{tomas_del_dia}")
    print(f"  Actores convocados ({len(actores_del_dia)}):
{actores_del_dia}")
    print()

```

```

Estado de la solución: Optimal
Costo Mínimo Total (Total de jornales/días pagados): 28

```

```

=== DÍA 1 ===
  Tomas a filmar (6/6): [1, 10, 11, 12, 22, 26]
  Actores convocados (8): [1, 2, 3, 4, 5, 6, 8, 9]

=== DÍA 2 ===
  Tomas a filmar (6/6): [3, 6, 7, 16, 25, 28]
  Actores convocados (6): [1, 2, 4, 5, 7, 10]

=== DÍA 3 ===
  Tomas a filmar (6/6): [2, 13, 17, 20, 27, 30]
  Actores convocados (4): [1, 3, 4, 5]

=== DÍA 4 ===
  Tomas a filmar (6/6): [4, 5, 8, 9, 15, 21]
  Actores convocados (6): [1, 2, 4, 6, 7, 8]

=== DÍA 5 ===
  Tomas a filmar (6/6): [14, 18, 19, 23, 24, 29]
  Actores convocados (4): [1, 3, 5, 6]

```

1.)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

2.)¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?.

```

import math

#
=====
=====
# CÁLCULO 1: Sin tener en cuenta las restricciones
#
=====
=====
# Tenemos 30 tomas y cada una se puede asignar libremente a cualquiera
de los 5 días.
# La fórmula es: (Número de días) ^ (Número de tomas) = 5^30

opciones_dias = 5
total_tomas = 30

sin_restricciones = opciones_dias**total_tomas

print("=== 1. OPCIONES SIN RESTRICCIONES ===")
print(f"Fórmula: 5^30")
print(f"Resultado exacto: {sin_restricciones}")
print(f"En notación científica: {sin_restricciones:.2e}\n")

```

```

#
=====
=====
# CÁLCULO 2: Teniendo en cuenta las restricciones (6 tomas exactas por
día)
#
=====
=====
# Queremos agrupar 30 elementos en 5 grupos de 6 elementos cada uno.
# Usamos el Coeficiente Multinomial:  $30! / (6! * 6! * 6! * 6! * 6!)$ 

tomas_por_dia = 6
num_dias = 5

# math.factorial(n) calcula el factorial de n (n!)
numerador = math.factorial(total_tomas) # 30!
denominador = math.factorial(tomas_por_dia) ** num_dias # (6!)^5

# Combinaciones brutas (considerando el Día 1, Día 2, etc., como
distintos)
combinaciones_con_orden_dias = numerador // denominador

# Si no nos importa el orden de los 5 días entre sí (los días son
intercambiables),
# dividimos entre 5! (las permutaciones de los 5 días):
combinaciones_sin_orden_dias = combinaciones_con_orden_dias //
math.factorial(
    num_dias
)

print("=== 2. OPCIONES CON RESTRICCIONES ===")
print(
    f"Combinaciones etiquetando los días ( $30! / (6!)^5$ ):
{combinaciones_con_orden_dias}"
)
print(f"En notación científica: {combinaciones_con_orden_dias:.2e}\n")

print(
    f"Combinaciones reales sin importar el orden de los días ( $30! /
((6!)^5 * 5!)$ ): {combinaciones_sin_orden_dias}"
)
print(f"En notación científica: {combinaciones_sin_orden_dias:.2e}")

=== 1. OPCIONES SIN RESTRICCIONES ===
Fórmula:  $5^{30}$ 
Resultado exacto: 931322574615478515625
En notación científica:  $9.31e+20$ 

=== 2. OPCIONES CON RESTRICCIONES ===
Combinaciones etiquetando los días ( $30! / (6!)^5$ ): 1370874167589326400

```

En notación científica: $1.37e+18$

Combinaciones reales sin importar el orden de los días ($30! / ((6!)^5 * 5!)$): 11423951396577720

En notación científica: $1.14e+16$

****3.)**Modelo para el espacio de soluciones (*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)**

El algoritmo oficial que utiliza se llama ramificación y acotamiento (Branch and Bound) se acomoda por que usa números enteros y se justifica con los siguientes pasos como se resuelve el problema

1.Poner los límites (Definir el marco):Se registran todas las reglas del problema (los costos, las capacidades máximas y las obligaciones).

1. Calcular una meta teórica (El mejor escenario) el algoritmo hace una estimación matemática rápida del "mejor resultado posible".
2. Dividir el problema en elecciones (Tomar caminos), el algoritmo toma una decisión individual y divide el problema en dos opciones (como un camino que se bifurca).
3. Corta caminos malos antes de tiempo (Descartar en bloque) a medida que el programa avanza por un camino, va sumando los resultados. Si nota que un camino se está volviendo muy costoso o malo antes de terminar, se detiene y lo abandona de inmediato.

Según el modelo para el espacio de soluciones ¿Cual es la función objetivo? ¿Es un problema de maximización o minimización?

La función objetivo es sumar todas las asistencias de todos los actores durante los 5 es decir sumar todos los unos de la tabla de asistencias y es un problema de MINIMIZACIÓN por se quiere que el dinero que se pague sea lo menor posible

Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

ALGORITMO FuerzaBruta_EstudioGrabacion:

CostoMinimo = INFINITO

MejorAgenda = NULO

1. Generar TODAS las particiones posibles de las 30 tomas en 5 días (donde cada toma pertenece a exactamente 1 día):

PARA CADA Combinacion C en TodasLasCombinacionesPosibles:

```
// Paso A: Verificar Restricción de Capacidad
SI NO (Cada día en C tiene EXACTAMENTE o MÁXIMO 6 tomas):
    CONTINUAR (Descartar esta combinación y pasar a la
siguiente)
```

```

// Paso B: Calcular Costo de la Combinación
CostoActual = 0
PARA CADA Día d del 1 al 5:
    PARA CADA Actor a del 1 al 10:
        SI el Actor a actúa en al menos una toma asignada al
Día d en C:
            CostoActual = CostoActual + 1 (Sumar 1 jornada a
pagar)

// Paso C: Actualizar la Mejor Solución Encontrada
SI CostoActual < CostoMinimo:
    CostoMinimo = CostoActual
    MejorAgenda = C

RETORNAR CostoMinimo, MejorAgenda

```

Explicación de los Pasos del Algoritmo

- **Generación Total:** Explora las 1.37×10^{18} formas distintas de repartir las 30 tomas en los 5 días sin usar ningún atajo.
- **Filtro de Reglas:** Verifica que el plan no ponga más de 6 tomas en un mismo día.
- **Cálculo de la Función Objetivo:** Revisa la lista de actores necesarios para las tomas de cada día y cuenta cuántos actores distintos tienen que ir a trabajar.
- **Selección del Óptimo:** Mantiene un registro del costo más bajo encontrado hasta el momento y la agenda que lo produjo.

```

import itertools

def fuerza_bruta_estudio(tomas, actores_por_toma, dias=5,
max_tomas_dia=6):
    num_tomas = len(tomas)
    costo_minimo = float('inf')
    mejor_agenda = None

    # Generar todas las asignaciones posibles de días para cada toma
    todas_las_combinaciones = itertools.product(range(1, dias + 1),
repeat=num_tomas)

    for combinacion in todas_las_combinaciones:
        # 1. Filtro: Respetar máximo 6 tomas por día
        conteo_dias = {d: combinacion.count(d) for d in range(1, dias
+ 1)}
        if any(cantidad > max_tomas_dia for cantidad in
conteo_dias.values()):
            continue

        # Agrupar tomas por día
        agenda_actual = {d: [] for d in range(1, dias + 1)}
        for id_toma, dia_asignado in zip(tomas, combinacion):

```



```

        agenda_actual[dia_asignado].append(id_toma)

# 2. Función Objetivo: Contar asistencias totales de actores
costo_actual = 0
for dia, tomas_del_dia in agenda_actual.items():
    actores_del_dia = set()
    for toma in tomas_del_dia:
        actores_del_dia.update(actores_por_toma[toma])
    costo_actual += len(actores_del_dia)

# 3. Guardar la mejor combinación
if costo_actual < costo_minimo:
    costo_minimo = costo_actual
    mejor_agenda = agenda_actual

return costo_minimo, mejor_agenda

# --- EJECUCIÓN E IMPRESIÓN DEL RESULTADO ---
if __name__ == "__main__":
    # Caso de prueba funcional de 6 tomas
    tomas = list(range(1, 30))
    actores_por_toma = {
        1: [1, 2],
        2: [2, 3],
        3: [1, 3],
        4: [4, 5],
        5: [5, 6],
        6: [4, 6]
    }

    costo_optimo, agenda_optima = fuerza_bruta_estudio(tomas,
    actores_por_toma)

    # Imprimir el resultado de forma estructurada
    print("=" * 45)
    print("    RESULTADO DE OPTIMIZACIÓN (FUERZA BRUTA)    ")
    print("=" * 45)
    print(f"Costo mínimo total (jornales a pagar): {costo_optimo}\n")
    print("Distribución de la agenda por día:")
    print("-" * 45)

    for dia, tomas_asignadas in agenda_optima.items():
        # Identificar actores que deben asistir ese día
        actores_dia = set()
        for toma in tomas_asignadas:
            actores_dia.update(actores_por_toma[toma])

        actores_str = ", ".join(map(str, sorted(actores_dia))) if
actores_dia else "Ninguno"

```

```

        tomas_str = ", ".join(map(str, tomas_asignadas)) if
tomas_asignadas else "Sin tomas"

    print(f"Día {dia}:")
    print(f"    • Tomas a grabar: [{tomas_str}]")
    print(f"    • Actores convocados: [{actores_str}]")
    print(f"({len(actores_dia)} pagos)")
    print("-" * 45)

```

Calcula la complejidad del algoritmo

1. Generación Total → La Cantidad de Iteraciones ($O(5^{30})$) Intenta crear las 1.37×10^{18} combinaciones de repartir las 30 tomas en los 5 días (o el total teórico de 5^{30}). Su impacto en la complejidad: Este paso representa la Complejidad Temporal Exponencial. Define la cantidad de veces que el bucle principal se va a repetir. Como cada toma nueva multiplica las opciones por 5, aquí es donde el tiempo de ejecución explota.
2. Filtro de Reglas → El Primer Trabajo Interno ($O(N)$) Cuenta las tomas asignadas a cada día para asegurar que no se superen las 6 tomas diarias. Su impacto en la complejidad: Para hacer esta validación, la computadora tiene que recorrer la lista de las 30 tomas. Es un trabajo lineal respecto al número de tomas ($N=30$).
3. Cálculo de la Función Objetivo → El Segundo Trabajo Interno ($O(D \cdot A \cdot N)$) Revisa día por día (5 días) la lista de los 10 actores para ver quiénes están convocados según las tomas asignadas a ese día. Su impacto en la complejidad: Requiere multiplicar los 5 días por los 10 actores y verificar las tomas correspondientes. Es el procesamiento más pesado que se ejecuta dentro de cada una de las 1.37×10^{18} iteraciones.
4. Selección del Óptimo → La Memoria Ahorradora ($O(N)$) Compara el costo de la combinación actual contra el récord mínimo y solo guarda la agenda si es mejor. Su impacto en la complejidad: Representa la Complejidad Espacial Lineal. Como no guarda las 1.37×10^{18} combinaciones en la memoria al mismo tiempo, sino que solo mantiene la mejor agenda actual de 30 tomas, la memoria RAM requerida es prácticamente cero.

Complejidad Temporal Total Multiplicando el número de combinaciones por el trabajo realizado dentro de cada iteración: "Complejidad Temporal Total" = $O(D^N \cdot (N + D \cdot A \cdot N))$ Simplificando términos dentro del paréntesis (donde $D \cdot A \cdot N$ predomina sobre N): $O(D^N \cdot D \cdot A \cdot N)$ Al sustituir por los valores de entrada y ver el comportamiento asintótico según crece el número de tomas N , el término exponencial D^N (o 5^N) domina por completo sobre los términos polinomiales. Complejidad Temporal Asintótica: $O(5^N)$ (Tiempo Exponencial). Complejidad Espacial El algoritmo almacena únicamente la combinación actual (C), el CostoMínimo y la MejorAgenda. Complejidad Espacial: $O(N)$ (Espacio Lineal), ya que solo necesita guardar en memoria la estructura de la agenda de N tomas.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

```

import random

# Fijar semilla para que los datos sean reproducibles (mismos datos
# cada vez que ejecutes)
random.seed(42)

```

```

NUM_TOMAS = 30
NUM_ACTORES = 10

tomas = list(range(1, NUM_TOMAS + 1))
actores = list(range(1, NUM_ACTORES + 1))

# Generar qué actores participan en cada toma (entre 1 y 3 actores por
toma)
actores_por_toma = {}
for toma in tomas:
    # Seleccionar una cantidad aleatoria de actores para la toma (de 1
a 3)
    num_actores_en_toma = random.randint(1, 3)
    # Seleccionar los actores específicos de forma aleatoria sin
repetir en la misma toma
    actores_asignados = sorted(random.sample(actores,
num_actores_en_toma))
    actores_por_toma[toma] = actores_asignados

# --- IMPRESIÓN DE RESULTADOS ---
print(f"JUEGO DE DATOS ALEATORIOS GENERADO ({NUM_TOMAS} TOMAS,
{NUM_ACTORES} ACTORES)\n")
print(f"Lista de Tomas: {tomas}\n")
print("Matriz de Actores por Toma:")
print("-" * 40)
for toma, lista_actores in actores_por_toma.items():
    print(f"Toma {toma:2d}: Actores {lista_actores}")

```

JUEGO DE DATOS ALEATORIOS GENERADO (30 TOMAS, 10 ACTORES)

Lista de Tomas: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Matriz de Actores por Toma:

```

-----
Toma 1: Actores [1, 2, 5]
Toma 2: Actores [4]
Toma 3: Actores [2]
Toma 4: Actores [2, 7, 9]
Toma 5: Actores [1]
Toma 6: Actores [4]
Toma 7: Actores [9]
Toma 8: Actores [1, 4, 9]
Toma 9: Actores [4, 7, 9]
Toma 10: Actores [5, 10]
Toma 11: Actores [3]
Toma 12: Actores [5, 6, 7]
Toma 13: Actores [4]
Toma 14: Actores [2, 10]
Toma 15: Actores [2, 6]

```

```

Toma 16: Actores [5, 10]
Toma 17: Actores [8]
Toma 18: Actores [2, 7, 10]
Toma 19: Actores [4, 5, 6]
Toma 20: Actores [1, 2, 4]
Toma 21: Actores [2, 4]
Toma 22: Actores [7]
Toma 23: Actores [6, 8]
Toma 24: Actores [6]
Toma 25: Actores [4, 5]
Toma 26: Actores [2, 3, 4]
Toma 27: Actores [8]
Toma 28: Actores [5, 9]
Toma 29: Actores [6]
Toma 30: Actores [4]

```

Aplica el algoritmo al juego de datos generado

```

import pulp

# 1. Definición de Parámetros
dias = range(1, 6)          # 5 días
tomas = range(1, 31)        # 30 tomas
actores = range(1, 11)      # 10 actores

# Matriz binaria de presencia (30 tomas x 10 actores)
matriz_asistencia = {
    1: [1, 1, 0, 0, 1, 0, 0, 0, 0, 0], 2: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    3: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0], 4: [0, 1, 0, 0, 0, 0, 1, 0, 1, 1],
    5: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0], 6: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
    7: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0], 8: [1, 0, 0, 1, 0, 0, 0, 0, 0, 1],
    9: [0, 0, 0, 1, 0, 0, 1, 0, 1, 0], 10: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
    11: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0], 12: [0, 0, 0, 0, 1, 1, 1, 0, 0, 0],
    13: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0], 14: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
    15: [0, 1, 0, 0, 0, 1, 0, 0, 0, 0], 16: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
    17: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0], 18: [0, 0, 1, 0, 0, 0, 1, 0, 0, 0],
    19: [0, 0, 0, 1, 1, 1, 0, 0, 0, 0], 20: [1, 2, 0, 1, 0, 0, 0, 0, 0, 0],
    21: [0, 1, 0, 1, 0, 0, 0, 0, 0, 0], 22: [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],

```

```

    23: [0, 0, 0, 0, 0, 1, 0, 1, 0, 0], 24: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
0],
    25: [0, 0, 0, 1, 1, 0, 0, 0, 0, 0], 26: [0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
0],
    27: [0, 0, 0, 0, 0, 0, 0, 1, 0, 0], 28: [0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
0],
    29: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0], 30: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
0]
}

```

2. Inicialización del Modelo

```
prob = pulp.LpProblem("Optimizacion_Agenda_Estudio", pulp.LpMinimize)
```

3. Variables de Decisión

$x[t, d] = 1$ si la toma t se graba el día d

```
x = pulp.LpVariable.dicts("Toma_Dia", [(t, d) for t in tomas for d in dias], cat=pulp.LpBinary)
```

$y[a, d] = 1$ si el actor a asiste el día d

```
y = pulp.LpVariable.dicts("Actor_Dia", [(a, d) for a in actores for d in dias], cat=pulp.LpBinary)
```

4. Función Objetivo: Minimizar el total de jornadas/pagos

```
prob += pulp.lpSum([y[a, d] for a in actores for d in dias])
```

5. Restricciones

R1: Cada toma debe grabarse exactamente un día

```
for t in tomas:
    prob += pulp.lpSum([x[t, d] for d in dias]) == 1
```

R2: Máximo 6 tomas por día

```
for d in dias:
    prob += pulp.lpSum([x[t, d] for t in tomas]) <= 6
```

R3: Vincular asistencia del actor con las tomas asignadas al día

```
for d in dias:
    for a in actores:
        for t in tomas:
            if matriz_asistencia[t][a - 1] == 1:
                prob += y[a, d] >= x[t, d]
```

6. Resolver el modelo

```
prob.solve(pulp.PULP_CBC_CMD(msg=0))
```

7. Mostrar Resultados Óptimos

```
print("=" * 50)
print(f"ESTADO DE LA SOLUCIÓN: {pulp.LpStatus[prob.status]}")
print(f"COSTO MÍNIMO TOTAL: {int(pulp.value(prob.objective))} jornales")
print("=" * 50)
```

```

for d in dias:
    tomas_dia = [t for t in tomas if pulp.value(x[t, d]) == 1]
    actores_dia = [a for a in actores if pulp.value(y[a, d]) == 1]
    print(f"\nDía {d} ({len(tomas_dia)} tomas):")
    print(f"    • Tomas: {tomas_dia}")
    print(f"    • Actores convocados ({len(actores_dia)}):
{actores_dia}")

```

```

=====
ESTADO DE LA SOLUCIÓN: Optimal
COSTO MÍNIMO TOTAL: 21 jornales
=====

```

Día 1 (6 tomas):

- Tomas: [15, 17, 23, 24, 27, 29]
- Actores convocados (3): [2, 6, 8]

Día 2 (6 tomas):

- Tomas: [2, 7, 8, 13, 20, 30]
- Actores convocados (3): [1, 4, 9]

Día 3 (6 tomas):

- Tomas: [4, 9, 11, 18, 21, 26]
- Actores convocados (6): [2, 3, 4, 7, 9, 10]

Día 4 (6 tomas):

- Tomas: [6, 12, 19, 22, 25, 28]
- Actores convocados (5): [4, 5, 6, 7, 9]

Día 5 (6 tomas):

- Tomas: [1, 3, 5, 10, 14, 16]
- Actores convocados (4): [1, 2, 5, 10]

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

Referencias Bibliográficas

1. **T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein**, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

Utilizado para el análisis de complejidad asintótica $O(5^N)$, la teoría de algoritmos de fuerza bruta y la evaluación de crecimiento exponencial.

2. **S. S. Skiena**, *The Algorithm Design Manual*, 3rd ed. Stony Brook, NY, USA: Springer, 2020.

Utilizado para la formulación de problemas de optimización combinatoria y modelos de asignación de recursos.

3. **S. Mitchell**, *PuLP: An Integer Linear Programming Modeling Library in Python*, University of Auckland, New Zealand. Disponible en: <https://coin-or.github.io/pulp/>

Utilizado para la formulación del modelo de Programación Lineal Entera (ILP), definición de variables binarias y ejecución del solver CBC.

4. **Gemini (Google AI)**, *Modelo de Lenguaje e Inteligencia Artificial Colaborativa para el Análisis de Algoritmos y Optimización*, Google, 2026.

Utilizado para la generación interactiva del conjunto de datos aleatorio (matriz binaria 30×10), desarrollo de los algoritmos en Python, análisis paso a paso de la complejidad temporal y diseño del modelo en PuLP.

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Si en lugar de 30 tomas tuviéramos 200 tomas y 50 actores, probar todas las combinaciones o esperar a que el modelo matemático perfecto responda ya no es opción.

Usar atajos inteligentes (Poda y Algoritmos Avanzados): En lugar de revisar agendas absurdas, el sistema debería descartar ramas enteras de opciones en cuanto vea que romperán las reglas o que ya son más caras que el mejor récord que tenemos.

Buscar soluciones "casi perfectas" (Metaheurísticas): Se pueden usar algoritmos inspirados en la naturaleza (como algoritmos genéticos) que no garantizan encontrar la solución 100% perfecta, pero encuentran una solución 98% óptima en apenas 3 segundos en lugar de tardar días.

